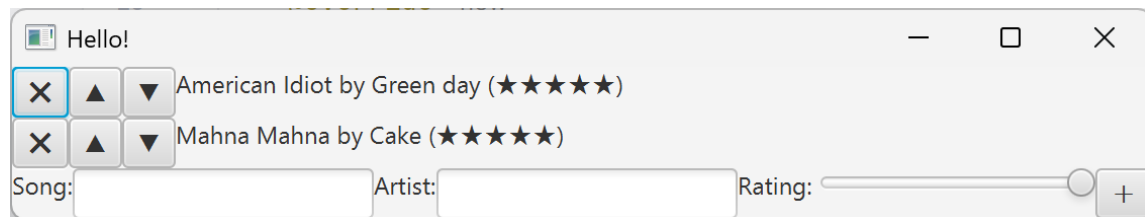


IC11: MVC - Back End



Today we will be collaborating with a partner (pair programming) to make a small fullstack application. One person is the front-end dev, the other the back-end dev. Pick a partner and pick your roles. You will each be contributing to the same repository, and pulling each other's work to get all of the code.

The Model and Controller

In this part of the exercise, you will build the **Model** for the playlist application. The Model represents the data and business logic, independent of the user interface. The Controller is the communication layer between the View and the Model. You will have to make these concurrently in order to test things as you go along.

1 The SongImpl Class

The **Song** interface models a single track in our playlist system. Your task is to create a concrete implementation, **SongImpl**, that reliably represents one song and safely handles the data provided by the user. The class should store essential song information and provide access to that information through the interface requirements.

- Implement the **Song** interface in a class named **SongImpl**.
- Store the song's title and artist as **String** values and the rating as an **int** from 0 to 5.
- Provide a constructor that accepts the title, artist, and rating, and use defensive programming to validate all inputs, including checking that the rating is within the allowed range.
- Implement all getter methods from the **Song** interface so that external code can access the stored values.

When finished, push to your git repo so that your frontend dev can pull it.

2 The ModelImpl Class

The `Model` interface represents the model that stores the collection of songs and supports the Observer pattern. Your task is to create a concrete implementation, `ModelImpl`, that maintains the playlist data and keeps track of all registered observers.

- Implement the `Model` interface in a class named `ModelImpl`.
- Maintain a list of songs and a list of observers.
- Provide a constructor that takes no parameters and begins with both lists empty.

Observer Methods

- `addObserver(ModelObserver observer)`: registers the given observer so that it will receive model updates.
- `removeObserver(ModelObserver observer)`: removes the given observer so that it will no longer receive model updates.
- `notifyObservers`: private method that informs all registered observers whenever the model changes. *Note: You will call this method whenever the state of the playlist changes (add, remove, shuffle).*

Getting the Model State:

Implement the following methods to interact with the underlying list:

- `getNumSongs()`: Return the size of the `playlist`.
- `getSong(int index)`: Return the `Song` object at the specific index.

Controller

Implement the methods in `ControllerImpl` as well, delegating them to the model.

Modifying the Model State:

Implement the methods that modify the playlist. **Crucial:** You must call `notifyObservers()` at the end of each of these methods so the View updates.

1. `addSong(Song song, int index)`: Insert the song into the `playlist` at the specified index using `playlist.add(index, song)`. Finally, call `notifyObservers()`.
2. `removeSong(int index)`: Remove the song at the specified index using `playlist.remove(index)`. Finally, call `notifyObservers()`.
3. `moveSong(int oldIdx, int newIdx)`: Moves the song within bounds. Must perform proper bounds check.
4. `shuffle()`: Use `Collections.shuffle(playlist)` to randomize the order. Finally, call `notifyObservers()`.

Controller

There is not a perfect match between the controller and model. The controller has the methods `moveSongUp()` and `moveSongDown()` that will call `moveSong()`.

Move Up and Move Down Logic

When the user attempts to move a song within the playlist, the controller must ensure that the requested action is valid. A song at the beginning of the playlist cannot be moved up, and a song at the end cannot be moved down. In these cases, the corresponding button should produce no change to the playlist.

- If the user selects “Move Up” on a song at index 0, the action must have no effect.
- If the user selects “Move Down” on the last song in the playlist, the action must have no effect.
- For all other cases, the controller must request that the model move the song to its new valid position.

Hint for `moveSong`

The logic for moving a song can be approached in two simple steps:

- Remove the song from its current index.
- Insert the same song back into the list at the new index.

After repositioning the song, the model must notify its observers so the views update correctly.

It is suggested to stop here and merge your work. Add some songs to your model, and see if the `PlayListView` can read and display them.

3 Submission

When you are finished, or when class ends (which ever comes first), commit your current changes to github, and submit what you have to gradescope!